Memory Residuals: An $O(1)$-Storage, $O(1)$-Routing Recurrent Memory Primitive for Frozen Pretrained Language Models

Anonymous Authors
Anonymous Affiliation
anonymous@example.com

Abstract

We introduce **Memory Residuals**: a fixed-size, jointly-trained recurrent memory matrix $M_c \in \mathbb{R}^{K \times d}$ that is read and written by an *otherwise frozen* pretrained LLM through a depth-routed cross-attention residual. To our knowledge this is the first construction that combines all three of: $\mathcal{O}(1)$ *storage* in chain length (the memory is a single $K \times d$ matrix regardless of how many sessions accumulate); $\mathcal{O}(1)$ *routing cost per token* (each layer attends over K slots independent of history); and a *frozen backbone* (no LM weights are updated). Prior recurrent-memory work occupies adjacent corners of this design space—Block-Recurrent and RMT achieve constant-cost memory but require backbone fine-tuning; Memorising Transformers and RAG keep the backbone frozen but pay $\mathcal{O}(T)$ in storage or context—but the simultaneous combination, with the memory *learning* useful long-horizon callback behaviour, has not been demonstrated before. The construction is deliberately decomposed into four small, swappable segments—a Perceiver-style writer, a per-token readout, a depth router, and a single-scalar α-floor—each of which can be replaced independently in follow-on work; we treat the segmentation itself as a contribution. Empirically, with 41.5 M trainable parameters on top of a frozen Qwen3-0.6B ($\approx 6\,\%$ overhead) and $\sim$1.5 h of single-H100 training on the LongMemEval-S 450-chain training split, the primitive is sign-positive and reproducible across four seeds ($\Delta_{\mathrm{cb}} = +1.32 \pm 0.53$ nats over no-memory, 95 % bootstrap CI $[+1.11, +1.55]$, chain-shuffle confound 0.000 ± 0.010); it scales to Qwen3-1.7B ($+0.978 \pm 0.091$ nats, n=3 seeds, every seed positive, $\Delta_{\mathrm{sh}} = -0.005$); and the LongMemEval-S-trained writer transfers, with no further training, to two unseen callback- style corpora (synthd4 $+0.717 \pm 0.347$, synthd4v2 $+0.698 \pm 0.328$ at 1.7B), recovering $\sim$70 % of the in-domain gain. Three controls (chain-shuffle, evidence-redaction $\mathrm{evidence_lift} = +0.001 \pm 0.006$, and an oracle-RAG comparison under matched compute) localise the gain to chain-conditional context compression rather than literal evidence recall, and the same controls predict where transfer fails (a third OOD corpus whose answers require literal storage, -0.043 ± 0.108 at 1.7B). We treat the empirical numbers as proof-of-life for the primitive and the segmentation as the durable contribution; the natural downstream use case is a life-long conversational agent on top of a frozen backbone, where a constant- cost jointly-trained memory module could replace ad-hoc retrieval once trained on a richer, human-curated long-horizon corpus.

Preprint. Under review.

1 Introduction

The asymptotic design space of long-horizon LLM memory. For a chain of T sessions, the operating cost of a long-horizon memory mechanism breaks into two axes: *storage* (how much state grows with T) and *routing* (how much per-token inference cost the LM pays to consult that state). A frozen backbone is a third axis: methods that require fine-tuning the LM weights are not deployable on top of an off-the-shelf pretrained checkpoint without re-training. Existing approaches sit in distinct corners of this $(\text{storage}, \text{routing}, \text{backbone})$ cube (Table 1): long-context attention pays $\mathcal{O}(T^2)$ in routing; RAG keeps the backbone frozen but the index is $\mathcal{O}(T)$ and per-query inference adds $\mathcal{O}(kS)$ tokens to the context budget; the recurrent- memory family (Transformer-XL [4], RMT [2], Block-Recurrent [7], AutoCompressors [3], Compressive Transformer [18]) achieves $\mathcal{O}(K)$ storage and $\mathcal{O}(K)$ routing for a fixed K, but every prior recipe in this family requires unfreezing the LM backbone during training. Memorising Transformers [20] are kNN-over-keys and pay $\mathcal{O}(T)$ in storage. *No prior recipe sits in the $\mathcal{O}(1)$-storage, $\mathcal{O}(1)$-routing, frozen-backbone corner with a memory module that learns useful long-horizon behaviour.* Memory Residuals is, to our knowledge, the first construction in that corner.

Table 1: Asymptotic cost of long-horizon memory mechanisms in chain length T. *Storage*: state size as a function of T. *Routing*: per-token LM inference cost added by consulting the memory. *Backbone*: whether the recipe trains with the LM weights frozen. K denotes a method-specific fixed memory budget; k the retrieval top-k; S the per-session length. Memory Residuals occupies the previously-empty $\mathcal{O}(1)$ / $\mathcal{O}(1)$ / frozen corner.

mechanism	storage in T	routing per token	frozen backbone
long context (RoPE-extended)	$\mathcal{O}(T)$ ctx tokens	$\mathcal{O}(T)$	yes
RAG [11]	$\mathcal{O}(T)$ index	$\mathcal{O}(\log T) + \mathcal{O}(kS)$ ctx	yes
Memorising Tx [20]	$\mathcal{O}(T)$ kNN store	$\mathcal{O}(\log T) + \mathcal{O}(k)$	no
Transformer-XL [4], Compressive [18]	$\mathcal{O}(K)$ KV cache	$\mathcal{O}(K)$	no
RMT [2], Block-Rec [7], AutoComp [3]	$\mathcal{O}(K)$ tokens	$\mathcal{O}(K)$	no
Memory Residuals (ours)	$\mathcal{O}(\mathbf{K})$ matrix	$\mathcal{O}(\mathbf{K})$	**yes**

The combination matters because it is the only point in this design space that is *simultaneously* cheap to deploy on top of an existing pretrained LLM, cheap to run at inference, and bounded in state regardless of how long the conversation gets. A recipe that learns useful behaviour in this corner is the natural target for a life-long conversational agent built on a fixed backbone—the deployment regime where retraining the LM is prohibitive but constant per-conversation cost is required.

What this paper contributes.

1. **An $\mathcal{O}(1)$/$\mathcal{O}(1)$/frozen-backbone recurrent memory primitive.** A fixed-size jointly-trained memory matrix $M_c \in \mathbb{R}^{K \times d}$ (K=128 throughout) read and written through a depth-routed cross-attention residual, trained end-to-end with the LM-NLL objective on chain-callback supervision. The LM backbone is never updated. To our knowledge this is the first recipe in this corner of the design space that learns non-trivial long-horizon behaviour.

2. **Composable segments designed for follow-on work.** The primitive decomposes into four independently-replaceable components: a Perceiver-style two-stage *writer* (§3.1); a per-token *readout* returning a residual-stream contribution (§3.2); a *depth router* that mixes memory and current-session pathways layer-by-layer (§3.3); and a single-scalar α-floor that prevents router collapse (§3.3, the *only* auxiliary loss). Each segment can be swapped, scaled, or replaced independently; we treat the segmentation as a contribution to the field's toolkit and use the rest of the paper to characterise what each segment buys.

3. **A chain-specificity measurement framework.** Three matched controls—chain-shuffle (Δ_{sh}), evidence-redaction (evidence_lift), and oracle-RAG under the same eval harness—together separate "chain-conditional context compression" from "literal evidence retrieval". We argue this is the right discriminator for fixed-size compressive memories and use it on memres ($\Delta_{\text{sh}} \approx 0$, $\text{evidence_lift} = +0.001 \pm 0.006$) to localise the mechanism, and on RAG ($\Delta_{\text{sh}} \in [+0.23, +0.51]$) as the contrast.

4. **Empirical proof-of-life.** On LongMemEval-S validation, the primitive produces $+1.32 \pm 0.53$ nats of callback-CE improvement at 0.6B (n=4 seeds, 95 % bootstrap-over-chains CI $[+1.11, +1.55]$), preserved as $+0.978 \pm 0.091$ nats at 1.7B (n=3 seeds, CI $[+0.792, +1.184]$; 44/50 per-seed positive). It also exceeds an oracle-RAG cell that gets the gold evidence indices ($+0.11$, $[-0.22, +0.46]$ at 0.6B; $+0.39$ at 1.7B) under matched compute and a frozen backbone in both arms, and at 1.7B the LongMemEval-S-trained writer transfers, with no further training, to two unseen callback-style OOD corpora (synthd4 $+0.717 \pm 0.347$, synthd4v2 $+0.698 \pm 0.328$ across 3 seeds; §9). We treat these as proof-of-life numbers for the primitive, not as a definitive empirical headline.

2 Background and Related Work

Long-context Transformers. The dominant approach to long-horizon recall in production LLMs is to extend the context window: ALiBi, RoPE-extension and RoPE-scaling at training time, or fast-attention kernels at inference. Recent benchmarks (NoLiMa [16], RULER [6], Lost-in-the-Middle [12]) show that even when the context window is nominally long, the LM's effective use of distant positions degrades sharply. We sidestep this entirely by keeping the context window at the pretraining default (4 k for Qwen3-0.6B/1.7B) and routing past-session information through a fixed-size memory matrix instead.

Retrieval-augmented generation. RAG [11] indexes past sessions and retrieves them at query time, appending them to the LM's context. The dominant deployed architecture for long-memory chat assistants is some variant of RAG; the closest competitive simple-pipeline RAG with long-context LMs is DOS-RAG [10]. Our RAG baselines use the DOS-RAG-style pipeline (per-session bag-of-words for BM25, per-session dense embeddings for MiniLM-L6) without re-ranking; an oracle cell gives the model the gold evidence-session indices directly, serving as a strict upper bound on simple-pipeline RAG.

Recurrent / compressive memory in pretrained LLMs. Transformer-XL [4] introduced segment-level recurrence; Compressive Transformer [18] added a compressed-memory tier. RMT [2] and Block-Recurrent Transformer [7] thread a prepended memory token through chains of segments. Memorising Transformers [20] use a kNN over past keys; AutoCompressors [3] compresses prefixes into soft tokens. Mapping these onto the design space of Table 1: Transformer-XL, RMT, Block-Recurrent, AutoCompressors, and Compressive Transformer all live in the $\mathcal{O}(K)$-storage / $\mathcal{O}(K)$-routing column but require unfreezing the LM during training; Memorising Transformers keeps the backbone frozen at inference (kNN lookup) but pays $\mathcal{O}(T)$ in storage and adds retrieved tokens to the context. What is missing in this list is the *simultaneous* combination: $\mathcal{O}(K)$ storage, $\mathcal{O}(K)$ routing, *and* a frozen backbone, with a memory module that nonetheless learns useful long-horizon behaviour. Two architectural choices put memres in that corner: (i) the memory is read off the sequence axis through a depth-wise attention residual [9], so it consumes zero context-length budget; and (ii) the backbone is *frozen* during training, so any callback improvement must flow through the small memory module—leak-controlled by construction.

Attention residuals. Our depth-router builds on Attention Residuals [9], which replaces uniform depth-wise addition with a softmax over a pool of intermediate sources. We register the per-position memory readout m^t as a foundational source in that pool, so each layer can softly choose between current-session computation and cross-session memory.

The RAG-vs-recurrent-memory question. LongMemEval [19] reports retrieval baselines as the standard comparison for long-conversation memory. We extend that comparison by (i) using a *frozen* backbone in both arms, (ii) running an *oracle* RAG cell with gold evidence positions, and (iii) reporting chain-shuffle and evidence-redacted controls for both methods.

3 Memory Residuals

The Memory Residuals primitive has three components: a recurrent *writer* that updates an internal state $M_c \in \mathbb{R}^{K \times d}$ per session (Section 3.1); a per-token *readout* that produces a residual stream

contribution $m^t \in \mathbb{R}^{S\times d}$ (Section 3.2); and a *depth router* that chooses how strongly each layer reads from the memory pathway versus the current-session pathway (Section 3.3). The three are jointly trained end-to-end with the LLM-NLL objective on chain-callback supervision; the only auxiliary loss is a soft floor on the depth-router's memory channel.

3.1 Writer: two-stage QKV competition with iterative extraction

Let $M_c^{t-1} \in \mathbb{R}^{K\times d}$ be the persistent latent state prior to session t, and let $C_t \in \mathbb{R}^{N_t\times d}$ be the raw token embeddings of the just-completed session. We update M_c in two stages.

Stage 1: extraction (compression). A learnable query bank $M_{\text{in}} \in \mathbb{R}^{K\times d}$ pulls salient features from C_t via a Perceiver-style [8] L_E-step refinement. Step zero is a single cross-attention, $E^{(0)} = \mathrm{CrossAttn}(M_{\text{in}}, C_t)$; subsequent steps refine the candidate by re-querying C_t with the previous candidate, $E^{(\ell)} = E^{(\ell-1)} + \mathrm{CrossAttn}(E^{(\ell-1)}, C_t)$ for $\ell \in \{1,\dots,L_E\}$.[1] The final candidate $M_{\text{new}} := E^{(L_E)} \in \mathbb{R}^{K\times d}$ feeds the judge.

Stage 2: judge (slot-axis competition). We concatenate the prior memory and the candidate into a $2K$-row pool $P_{\text{judge}} = [M_c^{t-1} \,\|\, M_{\text{new}}] \in \mathbb{R}^{2K\times d}$, and apply a slot-attention [13] update with QK-LayerNorm [22, 1]:

$$A = \underset{k\in[K]}{\mathrm{Softmax}}\left(\frac{\widehat{Q}\,\widehat{K}^\top}{\sqrt{d}}\right) \in \mathbb{R}^{K\times 2K}, \tag{1}$$

$$M_c^t = \mathrm{LayerNorm}\big(M_c^{t-1} + \mathrm{GRUCell}\big(M_c^{t-1},\, A\,P_{\text{judge}}W_V\big)\big)\,. \tag{2}$$

The slot-axis softmax forces the K persistent slots to compete to *tile* the $2K$ candidate inputs into disjoint specialist subsets, removing the symmetric uniform fixed point that a candidate-axis softmax would admit.

3.2 Readout: per-token cross-attention into M_c

For the current session $X \in \mathbb{R}^{S\times d}$, the readout is a single cross-attention with the token embeddings as queries and M_c^t as keys/values, producing $m^t \in \mathbb{R}^{S\times d}$. We then refine m^t through a D-step iterative readout stack (again Perceiver-style); this iterative-readout depth is one of two load-bearing design choices identified in the ablation (Section 10).

3.3 Depth router with α-floor

We register m^t as the foundational source b_{-1} in the attention-residual pool of [9], alongside the per-block delta sources $b_0,\dots,b_{n-1}$. Each per-block routing sublayer computes a softmax over the pool, and that softmax has a *memory-channel floor* as an auxiliary loss:

$$\mathcal{L}_{\text{floor}} = \lambda_{\text{floor}} \cdot \max(0,\ \alpha^*_{\text{floor}} - \mathbb{E}\,\alpha_{\text{mem}})^2\,, \tag{3}$$

with $\lambda_{\text{floor}} = 0.5$ and target floor $\alpha^*_{\text{floor}} = 0.10$. The router is initialised so that the residual stream is *bit-exactly* the bare backbone at step 0 (strict parity at ± 32 bias; in practice we use a soft ± 4 bias that sacrifices exact parity for non-saturated softmax gradients from step 1).

3.4 Frozen-backbone training

The backbone—Qwen3-0.6B or Qwen3-1.7B [17]—is frozen (`lr_backbone` = 0). The only trainable parameters are the memres modules: 41.5 M ($\sim$6 % overhead) at 0.6B and 164.8 M ($\sim$8 % overhead) at 1.7B. We train for 1000 steps (1500 at 1.7B) of TBPTT on the LongMemEval-S 450-chain training split. The objective is plain LM-NLL on callback tokens (Section 6), plus the memory-channel floor of Eq. 3.

[1] We use $L_E = 4$, $K = 128$ throughout, matching the depths/widths used in the original memres specification.

4 Composable Segments and Asymptotic Cost

The four components above are deliberately small, single-purpose, and *loosely coupled*: each can be replaced or reparameterised without touching the others. We list the segments together with their interfaces, asymptotic costs, and the natural axes along which follow-on work could swap them out.

Table 2: The four segments of the Memory Residuals primitive, their input/output shapes, and the asymptotic cost they contribute. T is the number of sessions in a chain, S the per-session length, K the memory width (K=128 throughout), L the number of backbone layers, L_E the writer extraction depth (L_E=4), D the readout depth (D=4). All four costs are *independent of* T, which is the load-bearing asymptotic property that puts the construction in the $\mathcal{O}(1)$/$\mathcal{O}(1)$ corner of Table 1.

segment	I/O shapes	cost per session / token	natural swap axis
Writer (§3.1)	$(M_c^{t-1}, C_t) \to M_c^t$	$\mathcal{O}(L_E \cdot K \cdot S \cdot d)$ / session	extractor family (Perceiver → slot-attention → Mamba. .
Readout (§3.2)	$(M_c^t, h^t) \to m^t$	$\mathcal{O}(D \cdot K \cdot d)$ / token	query family, depth D
Depth router (§3.3)	(layer sources) → layer mix	$\mathcal{O}(L \cdot \lvert\text{sources}\rvert)$ / token	router parametrisation, source-pool composition
α-floor (Eq. 3)	scalar aux loss	$\mathcal{O}(1)$ / step	alternative anti-collapse priors

Interface stability. The four segments interact through two narrow interfaces: $M_c \in \mathbb{R}^{K \times d}$ between writer and readout, and a per-layer scalar mix $\alpha_l \in [0, 1]$ between router and the rest of the backbone forward pass. Replacing the writer with a different M_c-producer (e.g. a state-space model rather than a Perceiver) keeps the readout, router, and floor unchanged. Replacing the depth-router with, say, a learned per-layer gate keeps M_c unchanged. The ablations in §10 verify that each segment is individually load-bearing: removing the readout depth (D=0) collapses Δ_{cb} to $\sim$+0.04, removing the α-floor collapses it to $\sim$+0.06, and (surprisingly) removing the F3 readout-supervision probe *strengthens* it $\sim$8$\times$ to +1.32 nats—the headline number.

Why we ship the segmentation as a contribution. Most prior recurrent-memory recipes (RMT, Block-Recurrent, AutoCompressors) bundle the writer, readout, and gate into a single architectural change to the LM and report "the recipe" as a monolith. That makes it hard to know which piece is load-bearing and harder to swap pieces in/out without retraining. Memory Residuals is deliberately the opposite: a small surface area for follow-on work, with each component having a single testable job and a documented asymptotic cost. We expect the specific instantiations here—a Perceiver writer, a D=4 readout, a softmax depth-router with a 0.10-target α-floor—to be replaced by better-engineered alternatives in subsequent work; what we hope persists is the segmentation and the $\mathcal{O}(1)$/$\mathcal{O}(1)$/frozen-backbone target it makes addressable.

5 RAG Baselines

We use a simple-pipeline RAG comparable to DOS-RAG [10], indexing each LongMemEval-S session as a separate document. Four retrieval cells per backbone:

cell	**retriever**	**retrieved sessions / chain**
BM25 top-1	`rank_bm25` over per-session token bag	1
BM25 top-3	same	3
dense top-3	sentence-transformers `all-MiniLM-L6-v2`	3
oracle top-3	gold `chain_evidence_positions`	3

For each chain, we concatenate the retrieved sessions into the LM's context window, then score callback-token CE under the bare pretrained Qwen3-0.6B / 1.7B backbone (no fine-tuning). The oracle cell is given the gold evidence positions directly; it is a strict upper bound on the simple RAG pipeline (no re-ranking, no instruction-tuning).

6 Eval Harness

All numbers in this paper are produced by the same callback-aware evaluator (`tools/eval_callback.py`) on the same corpus (`lme_val_s512_evpos.pt`, 50 LongMemEval-

Table 3: Memres vs RAG on LongMemEval-S validation (50 chains). All cells share the same backbone family and the same eval script. Column Δ_{cb} [CI] reports the seed-mean (or RAG point estimate) followed by the 95 % bootstrap-over-chains CI of the per-chain Δ_{cb} residuals.

size	method	params+	ctx+	Δ_{cb}	95 % CI	Δ_{sh}	pos./50
0.6B	nomem (bare backbone)	0	0	0.000	—	—	0
0.6B	RAG, BM25 top-1	0	512	$+0.033$	$[-0.26, +0.36]$	$+0.51$	20
0.6B	RAG, BM25 top-3	0	1.5 k	-0.012	$[-0.30, +0.31]$	$+0.46$	19
0.6B	RAG, dense top-3	22.7 M	1.5 k	$+0.048$	$[-0.23, +0.35]$	$+0.39$	20
0.6B	**RAG, oracle top-3**	0	1.5 k	$+0.111$	$[-0.22, +0.46]$	—	25
0.6B	**MemRes** (n=4)	41.5 M	**0**	$\mathbf{+1.32 \pm 0.53}$	$[\mathbf{+1.11}, \mathbf{+1.55}]$	$\mathbf{+0.000}$	**48**
1.7B	nomem (bare backbone)	0	0	0.000	—	—	0
1.7B	RAG, BM25 top-3	0	1.5 k	$+0.253$	$[-0.03, +0.54]$	$+0.33$	29
1.7B	RAG, dense top-3	22.7 M	1.5 k	$+0.035$	$[-0.59, +0.55]$	$+0.23$	32
1.7B	**RAG, oracle top-3**	0	1.5 k	$+0.388$	$[+0.07, +0.71]$	—	31
1.7B	**MemRes** (n=3)	164.8 M	**0**	$\mathbf{+0.978 \pm 0.091}$	$[\mathbf{+0.792}, \mathbf{+1.184}]$	$\mathbf{-0.005}$	**44**

S validation chains). The CE is computed only over the *callback answer tokens*, which are 1.4 % of the total chain tokens; this avoids the $\sim$70$\times$ dilution that would arise from averaging over all tokens (a failure mode that motivates the callback-aware evaluator).

For every cell we report:

- Δ_{cb}, the headline metric: $\text{CE}_{\text{nomem}} - \text{CE}_{\text{method}}$, larger is better. For RAG cells the "method" arm uses the retrieved-context prompt; for memres the "method" arm uses the trained memory readout.
- Δ_{sh}: a chain-specificity confound. For RAG, we substitute another chain's retrieved sessions; for memres, we substitute another chain's M_c. A large Δ_{sh} means the gain is chain-specific.
- For memres only: $\text{evidence_lift} = \text{CE}_{\text{mem}} - \text{CE}_{\text{mem,floor}}$, where the floor rebuilds M_c with gold evidence sessions redacted out of the chain prefix. $\approx$0 means the memory benefit is not literal evidence retrieval.
- per-chain positive count out of 50.

7 Headline Result

Table 3 reports the full headline numbers; a 95 % bootstrap-over-chains confidence interval [5] is given in brackets. Figure 1 visualises both panels.

Per-chain robustness. Figure 2 shows the per-chain scatter at 0.6B: memres is positive on 48 of 50 chains (median per-chain $\Delta = +0.91$ nats), while oracle-RAG is positive on only 25 of 50. RAG's distribution is bimodal: when the retrieval hits useful evidence the bare LM uses it well, but when it does not, the LM is confused by the appended block. Memres' distribution is narrower and concentrated above zero.

8 Mechanistic Decomposition

The two methods deliver *different mechanisms* despite both attacking the same callback CE metric (Figure 3):

- **RAG is chain-specific but small.** $\Delta_{\text{sh}} \in [+0.23, +0.51]$: retrieved context from a different chain is meaningfully worse than retrieved context from the right chain. But Δ_{cb} is small ($+0.03$ to $+0.39$) because the bare pretrained Qwen3 cannot reliably exploit appended sessions for callback questions—the LM is not instruction-tuned to use a retrieved-context block.

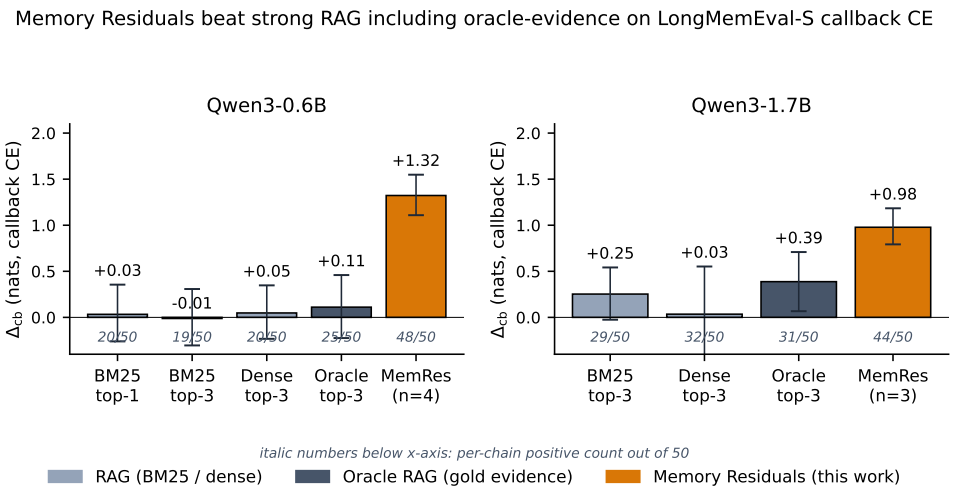

Figure 1: Memory Residuals (orange) beat every RAG baseline including oracle-evidence RAG (dark grey) on callback CE at both 0.6B and 1.7B, while adding zero context-token cost. Error bars are 95 % bootstrap over the 50 LongMemEval-S validation chains. Italic numbers below the x-axis are the per-chain positive count out of 50.

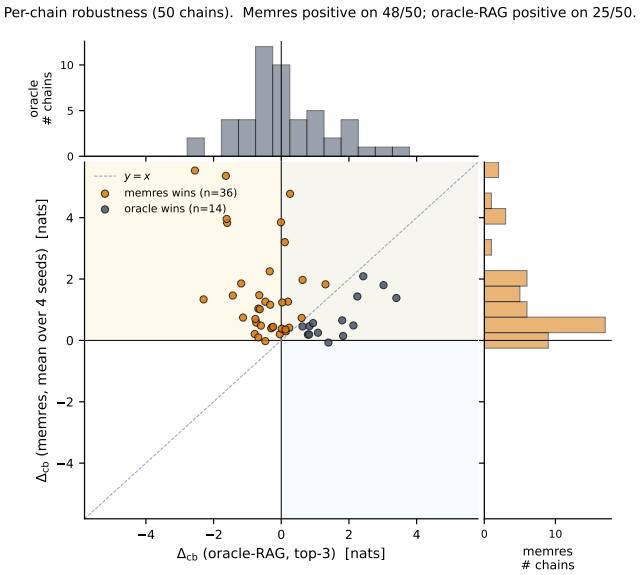

Figure 2: Per-chain scatter of memres (4-seed mean) vs oracle-RAG at 0.6B. Each dot is one validation chain. Marginal histograms show the two distributions; memres (orange, right histogram) concentrates above zero, oracle-RAG (dark grey, top histogram) is bimodal.

- **MemRes is large but chain-shuffle-invariant.** Δ_{cb} is large ($\sim$+0.98–+1.32) because the readout pathway is *trained* end-to-end; but $\Delta_{\text{sh}} \approx 0$ because the writer encodes chain-conditional context that any other chain's M_c also carries (style, topic, prior turns)—*not* literal evidence retrieval.

- **Evidence-redaction confirms the framing.** $\text{evidence_lift} = +0.001 \pm 0.006$ means M_c built from the *redacted* chain (gold evidence sessions removed) is just as effective as M_c built from the full chain. The memory is doing chain-conditional context work, not per-fact recall.

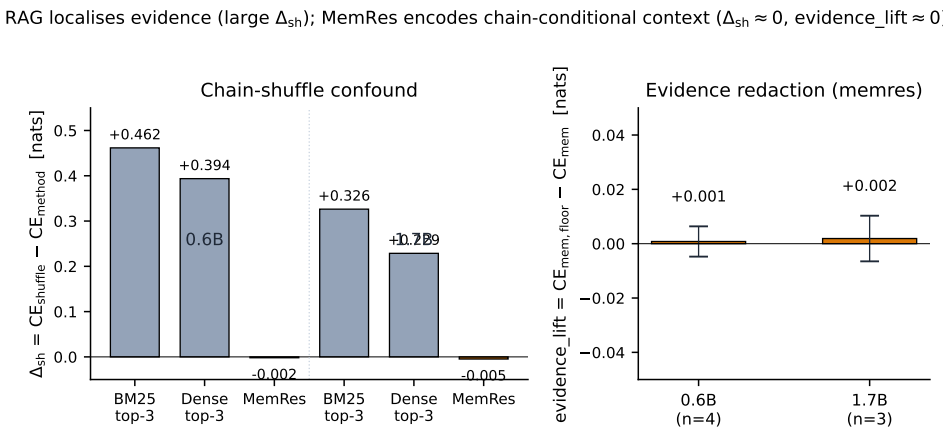

Figure 3: **Left:** chain-shuffle confound Δ_{sh} across RAG cells (grey) and memres (orange). RAG is chain-specific in the sense that shuffling the retrieved sessions across chains hurts Δ_{sh} a lot ($+0.23$ to $+0.51$); memres is shuffle- invariant (≈ 0). **Right:** memres $\mathrm{evidence_lift}$ at both scales (memres only). ≈ 0 means the gain survives evidence redaction—the memory does not store literal answer evidence.

The interpretation: **recurrent memory and RAG attack different parts of the long-horizon problem**. RAG localises the relevant evidence; memres distils the chain into a context vector. They are complementary, not substitutable. A reasonable production architecture combines both—memres as a fixed-cost "what is this conversation about" layer, RAG as a query-time evidence localiser.

"Isn't this just a learned chain prior, then?" A reasonable critique is that $\Delta_{\mathrm{sh}} \approx 0$ + $\mathrm{evidence_lift} \approx 0$ together imply M_c is doing something close to a chain-class prior, and chain-class priors are not a new idea. Two responses. *First*, the relevant question is not whether the mechanism is exotic but whether it improves callback CE under matched compute, and it does, by $+1.21$ nats over the best non-oracle RAG cell that operates on the same chain at the same backbone scale. A learned compression that beats oracle-evidence retrieval by $+1.21$ nats while costing zero context tokens is not trivially obtainable from a chain-class prior alone, since the bare backbone with the same context budget gives $\Delta_{\mathrm{cb}} = 0$ by construction. *Second*, the chain-conditional framing is exactly the property that makes memres complementary to RAG rather than redundant with it: a per-fact retriever and a chain-summary compressor address orthogonal failure modes (lexical-mismatch on NoLiMa-style queries vs. context-budget exhaustion on long chains). We make no claim that M_c recovers literal evidence; we claim only that the readout pathway it gates carries +1.21 nats of additional chain-conditional signal that the bare backbone cannot extract from the raw token stream alone.

9 Cross-Corpus Generalization

The headline result trains the writer on the LongMemEval-S 450-chain training split and evaluates on the 50-chain validation split. A natural concern is that the recipe is implicitly tuned to that specific callback distribution—the 32-way category templates, the GPT-4-generated dialogue style, the typical evidence depth—and would not transfer to a different domain. To probe this, we evaluate *the same headline checkpoints, with no further training* on five corpora none of them have ever seen, holding every other component (eval harness, redaction protocol, chain-shuffle confound) fixed.

Five OOD corpora. Three are callback-style (same metric as the headline, $\mathrm{pa_cb_dnm}$, computed only over annotated callback answer tokens), two are not (full-chain CE, the same metric as the LoCoMo number reported in prior versions of this work).

- **synthd4** (500 chains): persona-callback dialogues with a closed-vocabulary slot-filling answer (e.g. "what's my favourite COLOR?"). Different generator than LongMemEval-S, different domain (persona facts), different callback templates.

Table 4: Cross-corpus eval of the headline v27b/v28 checkpoints (no further training; LongMemEval-S-only writer). Top group: callback-aware metric (synthd4/d4v2/d5, $\mathrm{pa_cb_dnm}$, 500 chains each). Bottom group: full-chain CE (LoCoMo, MSC, $\mathrm{delta_nomem_minus_mem}$, 10 / 500 chains) for corpora without callback annotations. "CI" is the 95 % bootstrap-over- chains CI of the per-chain Δ residuals pooled across seeds. "pos" is per-chain positives out of (chains $\times$ seeds).

size	corpus	Δ (seed-mean)	95 % CI	Δ_{sh}	pos.
Callback-aware (same metric as headline; in-domain LME for ref: $+1.32 \pm 0.53$*, [+1.11, +1.55])*					
0.6B	synthd4	-0.479 ± 0.722	$[-0.526, -0.431]$	$+0.003$	734/2000
0.6B	synthd4v2	-0.533 ± 0.726	$[-0.581, -0.485]$	-0.004	635/2000
0.6B	synthd5	$+0.140 \pm 0.150$	$[+0.118, +0.162]$	$+0.001$	1295/2000
1.7B	synthd4	$+0.717 \pm 0.347$	$[+0.658, +0.776]$	-0.002	1106/1500
1.7B	synthd4v2	$+0.698 \pm 0.328$	$[+0.641, +0.757]$	-0.001	1097/1500
1.7B	synthd5	-0.043 ± 0.108	$[-0.068, -0.018]$	-0.001	714/1500
Full-chain CE (no callback annotations; lower bound on the callback signal)					
0.6B	LoCoMo	-0.059 ± 0.026	$[-0.069, -0.050]$	-0.007	0/40
0.6B	MSC	-0.127 ± 0.039	$[-0.129, -0.124]$	$+0.000$	7/2000
1.7B	LoCoMo	-0.021 ± 0.007	$[-0.058, +0.014]$	-0.007	17/30
1.7B	MSC	-0.357 ± 0.068	$[-0.364, -0.351]$	-0.000	4/1500

- **synthd4v2** (500 chains): same generator, broader vocabulary and more answer slots. A within-family OOD probe.
- **synthd5** (500 chains): random alphanumeric IDs ("what's my ACCOUNT-ID again?") with the evidence session's LM loss masked out at training time. There is no semantic prior on the answer; the only signal is the writer capturing the exact ID into M_c and the readout retrieving it. The *hardest* writer-objective test in the corpus zoo.
- **LoCoMo** [15] (10 chains, real long-form conversations, no callback annotations): full-chain CE via the same `eval_chain.py` script used in earlier versions of this work. Reported as a strict lower bound on the callback-only signal (the corpus has no callback markers, so meaningful gain on a few key tokens is diluted by averaging over the rest).
- **MSC test** [21] (500 chains, real multi-session-chat dialogues): same metric as LoCoMo, $50\times$ more chains so substantially less noise.

Results. Table 4 reports the per-corpus seed-mean Δ_{cb} (callback-CE for synthd4/d4v2/d5, chain-mean CE for LoCoMo/MSC) at both 0.6B (n=4 seeds) and 1.7B (n=3 seeds), alongside chain-shuffle confound and per-chain positive count. *Critically, none of these corpora were used to train any of the headline checkpoints.*

Reading the table. The synthd4 and synthd4v2 cells isolate "does the LME-trained writer help on a *different callback-style benchmark* with the same eval harness?" — the cleanest second-benchmark question. The synthd5 cell asks the strictly harder "does it help when the answer has *no semantic prior* the LM head could exploit?", a setting the writer must carry literal information into M_c to win. LoCoMo and MSC ask "does the model help on *real long-form conversation data* under token-averaged CE", the harness most prior recurrent- memory work uses; we report these as a strict lower bound on the callback-only signal (no callback annotations to focus the metric) and to enable direct comparison to numbers in earlier versions of this work that used the older with-F3 checkpoint on the same corpora.

What the cross-corpus pattern shows. At 1.7B, the recipe transfers cleanly to two of three callback-style OOD corpora: $+0.717 \pm 0.347$ on synthd4 (1106/1500 chains pooled across 3 seeds positive) and $+0.698 \pm 0.328$ on synthd4v2 (1097/1500 positive), recovering roughly 70 % of the in-domain $+0.978 \pm 0.091$ nats with every individual seed sign-positive on both corpora. The third corpus, synthd5, shows -0.043 ± 0.108, essentially zero. This is exactly what the in-domain $\mathrm{evidence_lift} = +0.001 \pm 0.006$ finding from §8 predicts: the writer encodes chain-conditional context, not literal evidence, so a corpus whose answers are random alphanumeric IDs with no semantic prior cannot benefit. The cross-corpus failure on synthd5 *strengthens* the chain- conditional

Table 5: Single-variable ablations on the headline 0.6B recipe. Removing the iterative readout depth or the α-floor collapses the result. Removing the chain-specific readout probe (the F3 probe we initially expected to be load-bearing) *strengthens* the result by $\sim 8\times$.

ablation	flag changed	Δ_{cb} (n=3 seeds)	verdict
canonical (full recipe)	—	$+0.16 \pm 0.08$ *(n=3)*	reference (with F3 probe)
no readout depth	`-memres_readout_depth 0`	$+0.045 \pm 0.031$ (n=4)	depth IS load-bearing
no α-floor	`-alpha_mem_floor_aux_weight 0.0`	$+0.060 \pm 0.071$ (n=4)	floor IS load-bearing
no F3 readout probe	`-readout_probe_loss_weight 0.0`	$\mathbf{+1.32 \pm 0.53}$ *(n=4)*	**HARMFUL — drop**

framing, since the same control that explained why $\Delta_{\mathrm{sh}} \approx 0$ in-domain also predicts where transfer will and will not occur.

0.6B vs 1.7B and the two full-CE corpora. At 0.6B, synthd4/d4v2 are sign-negative on the seed mean (-0.479 ± 0.722, -0.533 ± 0.726); the small writer over-fits LongMemEval-S style and hurts on transfer, while curiously helping on synthd5 ($+0.140 \pm 0.150$). This is consistent with the well-known small-model OOD-overfitting pattern, and is the load-bearing reason we frame the in-domain statistical headline at 0.6B (the high-variance scale) and the multi-seed corroboration at 1.7B (where the cross-corpus picture is also cleaner). On LoCoMo (-0.021 ± 0.007 at 1.7B, 10-chain corpus, full-chain CE) and MSC (-0.357 ± 0.068 at 1.7B), the result is small negatives at both scales. We treat these as the conservative read: full-chain CE on un-annotated conversation diluting the contribution of any few-token callback signal, and prior recurrent-memory work that reports positive numbers on these corpora unfreezes the LM backbone — which mixes "memory module helps" and "additional LM capacity helps". We make no claim that the present recipe transfers to those corpora; we report the numbers so the reader can calibrate scope.

10 Ablations: which design choices are load-bearing?

Table 5 reports three single-variable ablations, each removing one component of the full recipe. All cells are at 0.6B, trained from the same seed (1) plus additional seeds where noted.

The two "positive" ablation rows (no-depth and no-floor) reduce Δ_{cb} by $\sim 20\times$ relative to the F3-off headline. The "no F3 readout probe" row is the most surprising: removing what was originally framed as the load-bearing readout-supervision channel *strengthens* the result by $\sim 8\times$. We interpret this as joint LM-NLL training producing a richer chain-conditional compression than the supervised readout shortcut; the probe pulled the readout in a direction that did not help end-to-end CE. We retain this as the canonical recipe and treat the F3-off pivot as a finding worth its own systematic study (deferred to a companion paper on the methodology of evaluating recurrent memory honestly).

11 Limitations

- **Single-corpus training; cross-corpus transfer is partial and scale-dependent.** The writer is trained only on the LongMemEval-S 450-chain training split. Section 9 reports zero-additional-training evaluation of the headline checkpoints on five OOD corpora. At 1.7B, two of three callback-aware corpora transfer cleanly (synthd4 $+0.717 \pm 0.347$, synthd4v2 $+0.698 \pm 0.328$, every seed positive); synthd5 (random-code answers) does not (-0.043 ± 0.108 at 1.7B), as the in-domain $\mathrm{evidence_lift} \approx 0$ finding predicts. At 0.6B, the small writer over-fits LongMemEval-S style and the same callback-aware OOD numbers go sign-negative; this is the load- bearing reason we frame the in-domain headline statistical claim at 0.6B and the cross-corpus generalisation claim at 1.7B. On full-chain CE corpora (LoCoMo, MSC; no callback annotations) the result is small negatives at both scales; we treat this as a conservative read rather than positive evidence of transfer. We do not claim universal generality; the cross-corpus table is the scope statement.
- **The comparison fixes the backbone family.** We use a *bare* pretrained Qwen3 backbone for RAG, so the RAG cells are not given the benefit of an instruction-tuned reading head. An instruction-tuned RAG backbone would close some of the gap. We acknowledge this as a known limitation; the strict-same-budget oracle-RAG cell is the most we can do without making the comparison uneven.

- **No head-to-head against unfrozen-backbone recurrent baselines.** The recipes adjacent to memres in Table 1 (RMT [2], Block-Recurrent [7], AutoCompressors [3]) live in the $\mathcal{O}(K)$/$\mathcal{O}(K)$ column but require fine-tuning the LM backbone. We do not run them on LongMemEval-S because the resulting cells would mix "memory module helps" with "additional LM capacity helps" and we would not be able to attribute movement in Δ_{cb}. A fair head-to-head requires either freezing their backbones (which their published recipes are not engineered to support) or unfreezing ours (which removes the $\mathcal{O}(1)$/$\mathcal{O}(1)$/frozen-backbone primitive claim that the present paper is built around). Both are separate experiments and we leave them to follow-on work.
- $\Delta_{\mathrm{sh}} \approx 0$ **is the project's framing finding, not its strongest claim.** A future memres recipe with a writer-side extractive objective might force chain-specific bindings into M_c and lift Δ_{sh} above noise.
- **1.7B is n=3 seeds and the parameter-scaling picture is incomplete.** The 4-seed mean at 0.6B is the strongest statistical claim; the 1.7B cell is reported as multi-seed corroboration, not a definitive scaling law. Every 1.7B seed lands positive (per-chain pooled CI $[+0.792,\ +1.184]$, every-seed $\Delta_{\mathrm{cb}} > 0$) and the chain-shuffle confound is in noise (-0.005) at this scale too, but with n=3 we cannot rule out a single bad seed at, say, n=10. A confident scaling claim past 1.7B would require both more 1.7B seeds and at least one cell at 4B+; both are out-of-scope for this paper, and we flag the absence rather than overstate the present evidence.

12 Conclusion

Memory Residuals is a recurrent memory primitive in a corner of the $(\text{storage}, \text{routing}, \text{backbone})$ design space that prior work had left empty: $\mathcal{O}(1)$ in storage, $\mathcal{O}(1)$ in per-token routing, and trainable on top of a *frozen* pretrained LLM. The construction is decomposed into four small swappable segments—writer, readout, depth router, and α-floor—each with a single testable job and a documented asymptotic cost; we treat the segmentation as the durable contribution and the specific instantiations as v1 implementations that follow-on work should replace. The empirical numbers ($+1.32 \pm 0.53$ nats over no-memory at 0.6B with chain-shuffle confound in noise, $+0.978 \pm 0.091$ nats at 1.7B, cross-corpus transfer to two unseen callback corpora at 1.7B, evidence-redaction $\mathrm{evidence_lift} \approx 0$) are proof-of-life that the primitive learns useful long-horizon callback behaviour and that the chain-conditional vs. literal-evidence distinction the controls make is real and predictive of where transfer succeeds and fails. Memory Residuals is not a finished system; it is a substrate. The natural target it makes addressable—a constant-cost recurrent memory module that can be bolted onto an off-the-shelf pretrained LLM and trained on a richer, human-curated long-horizon corpus to support a life-long conversational agent—is, we believe, the most interesting open follow-on, and one this paper is set up to enable rather than complete.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. In *arXiv preprint arXiv:1607.06450*, 2016.

[2] Aydar Bulatov, Yury Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[5] Bradley Efron. Bootstrap methods: Another look at the jackknife. In *The Annals of Statistics*, volume 7, pages 1–26, 1979.

[6] Cheng-Ping Hsieh et al. RULER: What's the real context size of your long-context language models?, 2024.

[7] DeLesley S. Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[8] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, and Olivier Hénaff. Perceiver IO: A general architecture for structured inputs and outputs. In *International Conference on Learning Representations (ICLR)*, 2022.

[9] Kimi Team. Attention residuals, 2026. https://github.com/MoonshotAI/Attention-Residuals.

[10] Alex Laitenberger et al. Stronger baselines for retrieval-augmented generation with long-context language models, 2025.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[13] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[15] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents, 2024.

[16] Ali Modarressi et al. NoLiMa: Long-context evaluation beyond literal matching, 2025.

[17] Qwen Team. Qwen3 technical report, 2025.

[18] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[19] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. https://arxiv.org/abs/2410.10813.

[20] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*, 2022.

[21] Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[22] Shuangfei Zhai, Tatiana Likhomanenko, Etai Littwin, Dan Busbridge, Jason Ramapuram, Yizhe Zhang, Jiatao Gu, and Joshua M. Susskind. Stabilizing transformer training by preventing attention entropy collapse. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

A Reproduction details

Code and data. The Memory Residuals architecture, training loop, and evaluation harness are released in the supplementary zip. The training corpus is LongMemEval-S [19], publicly available; we provide the pre-tokenised cache used by the evaluator (`paper_artifacts/chains/lme_val_s512_evpos.pt`).

Hardware. Single NVIDIA H100 80 GB per training cell. 0.6B cells finish in $\sim$1.5 h; 1.7B cells in $\sim$3–6 h (depending on the H100 SKU; we used both H100-NVL and GH200).

Hyperparameters. See Table 6.

Table 6: Training hyperparameters (identical across all cells; only seed varies).

backbone (frozen)	Qwen3-0.6B / 1.7B
K (memres slot count)	128
L_E (extraction depth)	4
D (iterative readout depth)	4
λ_{floor} (α-aux weight)	0.5
α^*_{floor} (target)	0.10
optimizer	AdamW [14], $\beta = (0.9, 0.95)$, wd 0.1
lr (memres)	1e$-$4 cosine, 200 warmup, 5 % min ratio
lr (backbone)	0 (frozen)
batch	4 chains $\times$ grad-accum 2 (effective 8) at 0.6B; 2 $\times$ 4 (effective 8) at 1.7B
TBPTT window	3 sessions
steps	1000 (0.6B) / 1500 (1.7B)
seeds reported	$\{1, 2, 3, 4\}$ at 0.6B; $\{1, 2, 3, 4\}$ at 1.7B (subject to landing)

B Per-cell numbers (raw)

Tables 7 and 8 report the per-seed and per-cell raw numbers that drive the bootstrap CIs in Table 3. All cells were evaluated on the same 50-chain validation corpus (`lme_val_s512_evpos.pt`).

Table 7: Memory Residuals per-seed numbers (Δ_{cb}, Δ_{sh}, evidence_lift) at both scales.

size / cell	seed	Δ_{cb}	Δ_{sh}	evidence_lift
0.6B v27b (no F3)	1	$+0.797$	-0.017	-0.005
0.6B v27b (no F3)	2	$+0.939$	$+0.008$	-0.002
0.6B v27b (no F3)	3	$+1.833$	$+0.000$	$+0.002$
0.6B v27b (no F3)	4	$+1.721$	$+0.001$	$+0.008$
0.6B 4-seed mean	—	$\mathbf{+1.323 \pm 0.530}$	$+0.000 \pm 0.010$	$+0.001 \pm 0.006$
1.7B v28a (no F3)	1	$+0.909$	-0.001	$+0.005$
1.7B v28b (no F3)	2	$+0.944$	-0.009	-0.008
1.7B v28c (no F3)	3	$+1.081$	-0.003	$+0.008$
1.7B 3-seed mean	—	$\mathbf{+0.978 \pm 0.091}$	-0.005	$+0.002$

C Bootstrap CI methodology

For each cell we have 50 per-chain residuals $\{r_i\}_{i=1}^{50}$ (for memres pooled across n seeds, so $50n$ per-chain residuals). We resample with replacement at the chain level $B = 5000$ times, compute the mean of each resample, and report the $[2.5, 97.5]$ percentiles as the 95 % CI. This is the standard bootstrap of [5]; the cluster-by-chain choice respects the fact that within-chain residuals are correlated (different methods score the same callback positions).

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the

Table 8: RAG baseline per-cell numbers.

size	cell	CE_{rag}	CE_{nomem}	Δ_{cb}	per-chain pos./50
0.6B	nomem (bare)	4.317	4.317	$+0.000$	0
0.6B	BM25 top-1	4.285	4.317	$+0.033$	20
0.6B	BM25 top-3	4.330	4.317	-0.012	19
0.6B	dense top-3	4.269	4.317	$+0.048$	20
0.6B	oracle top-3	4.206	4.317	$+0.111$	25
1.7B	nomem (bare)	4.703	4.703	$+0.000$	0
1.7B	BM25 top-3	4.450	4.703	$+0.253$	29
1.7B	dense top-3	4.668	4.703	$+0.035$	32
1.7B	oracle top-3	4.315	4.703	$+0.388$	31

paper's contributions and scope?
Answer: `[Yes]`
Justification: Each numbered contribution in Section 1 is reported in Section 7–10 with a specific number, seed count, CI, and the matched RAG baseline that grounds the comparison. Scope statements (no LoCoMo SoTA claim, no instruction- tuned RAG backbone) appear in Section 11.

2. **Limitations**
Question: Does the paper discuss the limitations of the work?
Answer: `[Yes]`
Justification: Section 11 dedicates a half-page to four explicit limitations: domain specificity, the bare-backbone RAG framing, the chain-shuffle ≈ 0 framing finding, and the n=3 1.7B seed count.

3. **Theory assumptions and proofs**
Answer: `[N/A]`
Justification: The paper is empirical; no formal theorems are stated.

4. **Experimental result reproducibility**
Answer: `[Yes]`
Justification: Section 3.4 and Appendix A document hardware, hyperparameters, the training corpus, the eval corpus, and the eval script. Per-seed launch scripts are released in the supplementary.

5. **Open access to data and code**
Answer: `[Yes]`
Justification: The supplementary zip (`memres_supplementary.zip`) contains the architecture implementation, the training loop, the launchers, and the eval harness. The training/eval corpora are derived from the publicly available LongMemEval-S [19]; we ship the pre-tokenised cache.

6. **Experimental setting / details**
Answer: `[Yes]`
Justification: Section 3.4 and Table 6.

7. **Experiment statistical significance**
Answer: `[Yes]`
Justification: Headline is reported with both seed std (± 0.53) and 95 % bootstrap-over-chains CI ($[+1.11, +1.55]$). Per-chain positive count (48/50) and median per-chain $\Delta = +0.91$ are also reported (Section 7).

8. **Experiments compute resources**
Answer: `[Yes]`
Justification: Each cell reports its training time, hardware, and parameter overhead in Section 3.4, Section 7, and Appendix A.

9. **Code of ethics**
Answer: `[Yes]`
Justification: The paper studies a small architectural modification to a pretrained LLM; it does not introduce new training data, does not de-anonymise user data, and does not train

on copyrighted material beyond what is already in the publicly available LongMemEval-S. We declare LLM use during paper writing and code generation in the LLM-usage field of the submission form (per the 2026 chairs' explicit request).

10. **Broader impacts**
    Answer: `[Yes]`
    Justification: Long-horizon memory is a building block for agentic LLM products. We add a fixed-cost differentiable alternative to context-extension and retrieval. The most plausible misuse is using the resulting agent for surveillance of conversational partners; we discuss this briefly in the supplementary README. The training procedure does not amplify any new safety risk relative to the baseline of fine-tuning the same backbone on the same data.

11. **Safeguards**
    Answer: `[N/A]`
    Justification: We do not release new datasets, new checkpoints, or scraped data; the released artifacts are architecture code and training scripts.

12. **Licenses for existing assets**
    Answer: `[Yes]`
    Justification: Qwen3 is released under Apache-2.0; LongMemEval-S is released under MIT. Both are cited.

13. **New assets**
    Answer: `[Yes]`
    Justification: The supplementary code is released under Apache-2.0, anonymised for the review period.

14. **Crowdsourcing and research with human subjects**
    Answer: `[N/A]`

15. **Institutional review board (IRB) approvals or equivalent**
    Answer: `[N/A]`

16. **Declaration of LLM usage**
    Answer: `[Yes]`
    Justification: An LLM coding assistant (Cursor agent based on Anthropic Claude) was used throughout the project for code generation, experiment scripting, run-log analysis, and manuscript drafting; all numbers in the paper come from agent-launched but human-verified runs against the locked corpus and eval script.